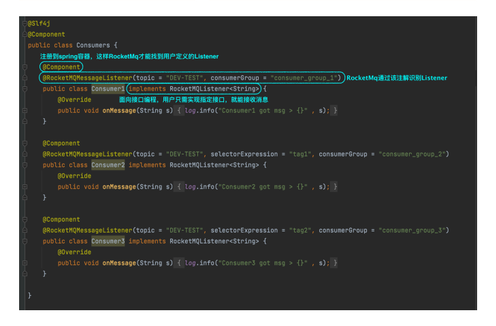
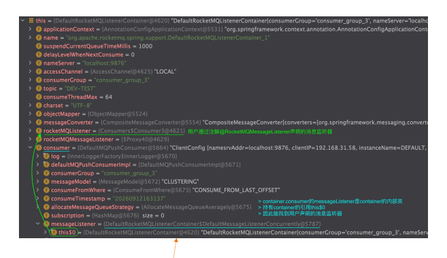
启动roketmq
> cd /Library/rocketmq-all-4.8.0-bin-release/bin
> nohup sh ./mqnamesrv &
> nohup sh ./mqbroker -n localhost:9876 &
> cat nohup.outcat nohup.out

关闭roketmq
> sh ./mqshutdown broker
> sh ./mqshutdown namesrv

RocketMQ怎么保证消息不丢?
> 生产者（同步发送+重试）：发送消息后，同步等待Broker确认，若失败自动重试，确保消息送达
> Broker（同步刷盘）：消息写入内存后，立刻刷入磁盘，避免宕机时内存消息丢失
> 消费者 1).确认&重试：SDK会确认业务逻辑处理消息的结果，如果没异常，才算消费成功
如果业务逻辑抛异常，SDK会将失败的那条msg同步发回broker重试
2).重试+死信队列：Broker默认最多重试16次，仍失败则转入死信队列，需人工处理
3).幂等处理，基于消息唯一ID实现幂等，避免重复消费

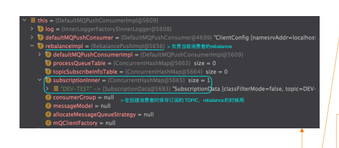


```java
@Slf4j
@Component
public class Consumers {
    注册到spring容器，这样RocketMq才能找到用户定义的Listener
    @Component
    @RocketMQMessageListener(topic = "DEV-TEST", consumerGroup = "consumer_group_1") RocketMq通过该注解识别Listener
    public class Consumer1 implements RocketMQListener<String> {
        @Override        面向接口编程，用户只需实现指定接口，就能接收消息
        public void onMessage(String s) { log.info("Consumer1 got msg > {}" , s); }
    }

    @Component
    @RocketMQMessageListener(topic = "DEV-TEST", selectorExpression = "tag1", consumerGroup = "consumer_group_2")
    public class Consumer2 implements RocketMQListener<String> {
        @Override
        public void onMessage(String s) { log.info("Consumer2 got msg > {}" , s); }
    }

    @Component
    @RocketMQMessageListener(topic = "DEV-TEST", selectorExpression = "tag2", consumerGroup = "consumer_group_3")
    public class Consumer3 implements RocketMQListener<String> {
        @Override
        public void onMessage(String s) { log.info("Consumer3 got msg > {}" , s); }
    }

}
```



```
this = {DefaultRocketMQListenerContainer@4620} "DefaultRocketMQListenerContainer{consumerGroup='consumer_group_3', nameServer='localho
  applicationContext = {AnnotationConfigApplicationContext@5531} "org.springframework.context.annotation.AnnotationConfigApplicationCont
  name = "org.apache.rocketmq.spring.support.DefaultRocketMQListenerContainer_1"
  suspendCurrentQueueTimeMillis = 1000
  delayLevelWhenNextConsume = 0
  nameServer = "localhost:9876"
  accessChannel = {AccessChannel@4625} "LOCAL"
  consumerGroup = "consumer_group_3"
  topic = "DEV-TEST"
  consumeThreadMax = 64
  charset = "UTF-8"
  objectMapper = {ObjectMapper@5524}
  messageConverter = {CompositeMessageConverter@5554} "CompositeMessageConverter[converters=[org.springframework.messaging.converte
  rocketMQListener = {Consumers$Consumer3@4621} 用户通过注解@RocketMQMessageListener声明的消息监听器
  rocketMQMessageListener = {$Proxy40@4629}
  consumer = {DefaultMQPushConsumer@5664} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.58, instanceName=DEFAULT,
    log = {InnerLoggerFactory$InnerLogger@5670}
    defaultMQPushConsumerImpl = {DefaultMQPushConsumerImpl@5671}
    consumerGroup = "consumer_group_3"
    messageModel = {MessageModel@5672} "CLUSTERING"
    consumeFromWhere = {ConsumeFromWhere@5673} "CONSUME_FROM_LAST_OFFSET"
    consumeTimestamp = "20260912163137"
    allocateMessageQueueStrategy = {AllocateMessageQueueAveragely@5675}
    subscription = {HashMap@5676}  size = 0
    messageListener = {DefaultRocketMQListenerContainer$DefaultMessageListenerConcurrently@5787}
      this$0 = {DefaultRocketMQListenerContainer@4620} "DefaultRocketMQListenerContainer{consumerGroup='consumer_group_3', nameServ
```

> container.consumer的messageListener是container的内部类
> 持有container的引用this$0
> 因此能找到用户声明的消息监听器

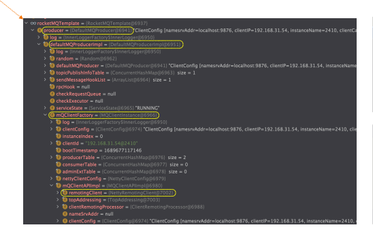


```
this = {DefaultMQPushConsumerImpl@5609}
  log = {InnerLoggerFactory$InnerLogger@5608}
  defaultMQPushConsumer = {DefaultMQPushConsumer@4699} "ClientConfig [namesrvAddr=localhost
  rebalanceImpl = {RebalancePushImpl@5656} > 负责当前消费者的rebalance
    defaultMQPushConsumerImpl = {DefaultMQPushConsumerImpl@5609}
    processQueueTable = {ConcurrentHashMap@5663}  size = 0
    topicSubscribeInfoTable = {ConcurrentHashMap@5664}  size = 0
    subscriptionInner = {ConcurrentHashMap@5665}  size = 1
      "DEV-TEST" -> {SubscriptionData@5683} "SubscriptionData [classFilterMode=false, topic=DEV-
    consumerGroup = null > 在创建消费者时保存订阅的 TOPIC，rebalance的时候用
    messageModel = null
    allocateMessageQueueStrategy = null
    mQClientFactory = null
```

SpringBoot + RocketMq 启动流程
MicroService3Application.main() -> 创建SpringApplication实例，执行run()
> prepareEnvironment
> 创建ApplicationContext: AnnotationConfigServletWebServerApplicationContext
> prepareContext
> refreshContext:
  > invokeBeanFactoryPostProcessors 自动装配RocketMq配置文件
  > registerBeanPostProcessors
  > onRefresh
  > finishBeanFactoryInitialization 注册RocketMQListener
  > finishRefresh

pom引入rocketmq-spring-boot-starter依赖，自动装配RocketMQAutoConfiguration时注册
rocketMQTemplate和ListenerContainerConfiguration的beanDefinition

创建完ListenerContainerConfiguration实例后，执行其afterSingletonsInstantiated方法
> 找出beanFactory中带@RocketMQMessageListener的beans
> 给每个listenerBean创建DefaultRocketMQListenerContainer container
  > createBeanInstance: 创建实例container，并设置rocketMQListener(listenerBean)，messageConverter等属性
  > afterPropertySet: initRocketMQPushConsumer() 创建消费者DefaultMQPushConsumer consumer
    consumer.setMessageListener(container内部类实例) consumer.subscribe(用户指定的TOPIC)
> container.start() → consumer.start()启动消费者



```
rocketMQTemplate = {RocketMQTemplate@6937}
  producer = {DefaultMQProducer@6941} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.54, instanceName=2410, clientCa
    log = {InnerLoggerFactory$InnerLogger@6950}
    defaultMQProducerImpl = {DefaultMQProducerImpl@6951}
      log = {InnerLoggerFactory$InnerLogger@6950}
      random = {Random@6962}
      defaultMQProducer = {DefaultMQProducer@6941} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.54, instanceName
      topicPublishInfoTable = {ConcurrentHashMap@6963}  size = 1
      sendMessageHookList = {ArrayList@6964}  size = 1
      rpcHook = null
      checkRequestQueue = null
      checkExecutor = null
      serviceState = {ServiceState@6965} "RUNNING"
      mQClientFactory = {MQClientInstance@6966}
        log = {InnerLoggerFactory$InnerLogger@6950}
        clientConfig = {ClientConfig@6974} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.54, instanceName=2410, cli
        instanceIndex = 0
        clientId = "192.168.31.54@2410"
        bootTimestamp = 1689677117146
        producerTable = {ConcurrentHashMap@6976}  size = 2
        consumerTable = {ConcurrentHashMap@6977}  size = 0
        adminExtTable = {ConcurrentHashMap@6978}  size = 0
        nettyClientConfig = {NettyClientConfig@6979}
        mQClientAPIImpl = {MQClientAPIImpl@6980}
          remotingClient = {NettyRemotingClient@7002}
          topAddressing = {TopAddressing@7003}
          clientRemotingProcessor = {ClientRemotingProcessor@6988}
          nameSrvAddr = null
        clientConfig = {ClientConfig@6974} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.54, instanceName=2410,
```

DefaultMQPushConsumerImpl.start() {
//1.校验消费者配置
//2.拷贝订阅关系
//3.创建MQClientInstance
//4.设置RebalanceImpl
//5.创建PullAPIWrapper，消息拉取核心类
//6.加载消费进度(Local)
//7.启动ConsumeMessageService
//8.注册消费者
//9.启动MQClientInstance
//10.拉取订阅的Topic路由信息
//11.SQL表达式上传到Broker编译
//12.给Broker发送心跳
//13.MQClientInstance rebalance一次
}

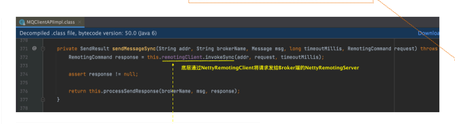
//启动一些定时执行的任务：给broker发送心跳，从NameServer拉取topic路由信息等

//MQClientInstance是JVM进程级别的，只会在创建首个消费者时创建

//将当前group存入MQClientInstance.consumerTable

MQClientInstance.start() { //如果已经start过，会直接返回
this.mQClientAPIImpl.fetchNameServerAddr();
this.mQClientAPIImpl.start(); //启动Netty客户端，负责和broker通信
this.startScheduledTask();
this.pullMessageService.start(); //启动pullMessage线程
this.rebalanceService.start(); //rebalance一次，开始拉取消息
this.defaultMQProducer.getDefaultMQProducerImpl().start(false);
this.serviceState = ServiceState.RUNNING;
}

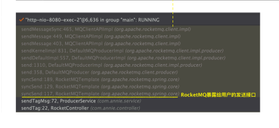
//上报Consumer Group、Topic、Tag、实例信息



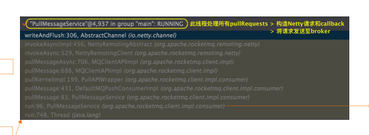
MQClientAPIImpl.class

Decompiled .class file, bytecode version: 50.0 (Java 6)

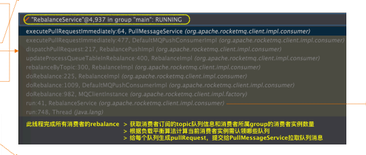
```java
private SendResult sendMessageSync(String addr, String brokerName, Message msg, long timeoutMillis, RemotingCommand request) throws
    RemotingCommand response = this.remotingClient.invokeSync(addr, request, timeoutMillis);
                                       底层通过NettyRemotingClient将请求发给Broker端的NettyRemotingServer
    assert response != null;

    return this.processSendResponse(brokerName, msg, response);
}
```



```
"http-nio-8080-exec-2"@6,636 in group "main": RUNNING
sendMessageSync:465, MQClientAPIImpl (org.apache.rocketmq.client.impl)
sendMessage:449, MQClientAPIImpl (org.apache.rocketmq.client.impl)
sendMessage:403, MQClientAPIImpl (org.apache.rocketmq.client.impl)
sendKernelImpl:831, DefaultMQProducerImpl (org.apache.rocketmq.client.impl.producer)
sendDefaultImpl:557, DefaultMQProducerImpl (org.apache.rocketmq.client.impl.producer)
send:1310, DefaultMQProducerImpl (org.apache.rocketmq.client.impl.producer)
send:358, DefaultMQProducer (org.apache.rocketmq.client.producer)
syncSend:189, RocketMQTemplate (org.apache.rocketmq.spring.core)
syncSend:129, RocketMQTemplate (org.apache.rocketmq.spring.core)
syncSend:117, RocketMQTemplate (org.apache.rocketmq.spring.core) RocketMQ暴露给用户的发送接口
sendTagMsg:72, ProducerService (com.annie.service)
sendTag:22, RocketController (com.annie.controller)
```

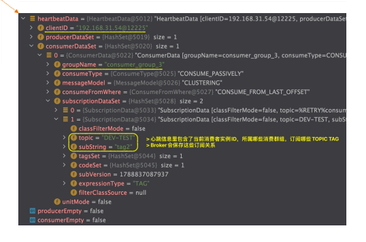


```
"PullMessageService"@4,937 in group "main": RUNNING 此线程处理所有pullRequests > 构造Netty请求和callback
                                                                               > 将请求发送至broker
writeAndFlush:306, AbstractChannel (io.netty.channel)
invokeAsyncImpl:456, NettyRemotingAbstract (org.apache.rocketmq.remoting.netty)
invokeAsync:529, NettyRemotingClient (org.apache.rocketmq.remoting.netty)
pullMessageAsync:706, MQClientAPIImpl (org.apache.rocketmq.client.impl)
pullMessage:688, MQClientAPIImpl (org.apache.rocketmq.client.impl)
pullKernelImpl:199, PullAPIWrapper (org.apache.rocketmq.client.impl.consumer)
pullMessage:431, DefaultMQPushConsumerImpl (org.apache.rocketmq.client.impl.consumer)
pullMessage:83, PullMessageService (org.apache.rocketmq.client.impl.consumer)
run:96, PullMessageService (org.apache.rocketmq.client.impl.consumer)
run:748, Thread (java.lang)
```



```
"RebalanceService"@4,937 in group "main": RUNNING
executePullRequestImmediately:64, PullMessageService (org.apache.rocketmq.client.impl.consumer)
executePullRequestImmediately:477, DefaultMQPushConsumerImpl (org.apache.rocketmq.client.impl.consumer)
dispatchPullRequest:217, RebalancePushImpl (org.apache.rocketmq.client.impl.consumer)
updateProcessQueueTableInRebalance:400, RebalanceImpl (org.apache.rocketmq.client.impl.consumer)
rebalanceByTopic:300, RebalanceImpl (org.apache.rocketmq.client.impl.consumer)
doRebalance:225, RebalanceImpl (org.apache.rocketmq.client.impl.consumer)
doRebalance:1009, DefaultMQPushConsumerImpl (org.apache.rocketmq.client.impl.consumer)
doRebalance:982, MQClientInstance (org.apache.rocketmq.client.impl.factory)
run:41, RebalanceService (org.apache.rocketmq.client.impl.consumer)
run:748, Thread (java.lang)
```

此线程完成所有消费者的rebalance > 获取消费者订阅的topic队列信息和消费者所属group的消费者实例数量
> 根据负载均衡算法计算当前消费者实例要认领哪些队列
> 给每个队列生成pullRequest，提交给PullMessageService拉取队列消息



```
heartbeatData = {HeartbeatData@5012} "HeartbeatData [clientID=192.168.31.54@12225, producerDataSe
  clientID = "192.168.31.54@12225"
  producerDataSet = {HashSet@5019}  size = 1
  consumerDataSet = {HashSet@5020}  size = 1
    0 = {ConsumerData@5022} "ConsumerData [groupName=consumer_group_3, consumeType=CONSU
      groupName = "consumer_group_3"
      consumeType = {ConsumeType@5025} "CONSUME_PASSIVELY"
      messageModel = {MessageModel@5026} "CLUSTERING"
      consumeFromWhere = {ConsumeFromWhere@5027} "CONSUME_FROM_LAST_OFFSET"
      subscriptionDataSet = {HashSet@5028}  size = 2
        0 = {SubscriptionData@5033} "SubscriptionData [classFilterMode=false, topic=%RETRY%consum
        1 = {SubscriptionData@5034} "SubscriptionData [classFilterMode=false, topic=DEV-TEST, subS
          classFilterMode = false
          topic = "DEV-TEST"
          subString = "tag2"
          tagsSet = {HashSet@5044}  size = 1
          codeSet = {HashSet@5045}  size = 1
          subVersion = 1788837087937
          expressionType = "TAG"
          filterClassSource = null
      unitMode = false
  producerEmpty = false
  consumerEmpty = false
```

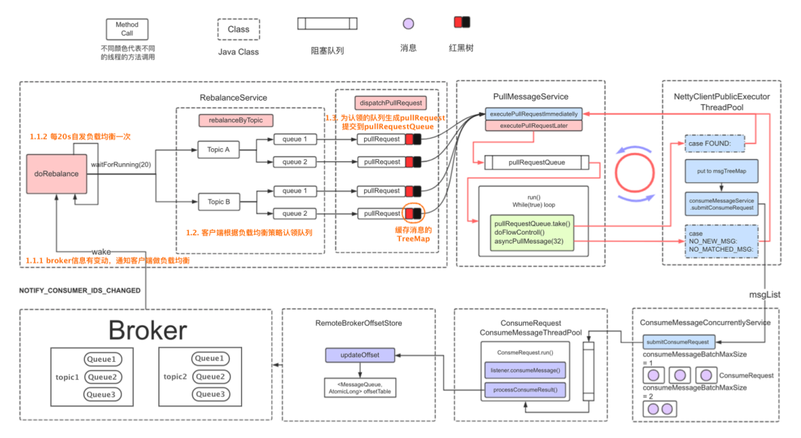
> 心跳信息里包含了当前消费者实例ID，所属哪些消费群组，订阅哪些 TOPIC TAG
> Broker会保存这些订阅关系

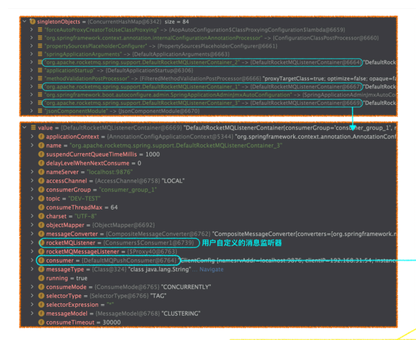
```java
void PullMessageService.run() { //线程一直run
    while(!isStopped()) {
        PullRequest pullRequest = (PullRequest)pullRequestQueue.take();
        pullMessage(pullRequest);
    }
}

void pullMessage(PullRequest pullRequest) {
    //根据pullRequest.consumerGroup找到消费者
    MQConsumerInner consumer = this.mQClientFactory.selectConsumer(pullRequest.getConsumerGroup());
    if (consumer != null) {
        DefaultMQPushConsumerImpl impl = (DefaultMQPushConsumerImpl)consumer;
        impl.pullMessage(pullRequest);
    }
}
```

```java
void RebalanceService.run() { //线程一直run
    while(!isStopped()) {
        waitForRunning(waitInterval); //默认每隔20s rebalance一次
        mqClientFactory.doRebalance();
    }
}

void MQClientInstance.doRebalance() {
    for (Map.Entry<String, MQConsumerInner> entry : consumerTable.entrySet()) {
        MQConsumerInner impl = entry.getValue();
        if (impl != null) impl.doRebalance(); //消费者做一次负载均衡
    }
}
```

> 在分布式消息中间件中，消费端的水平扩展和负载均衡决定了整个系统的吞吐边界
> rebalance的本质 - 多对多的动态映射
一个TOPIC包含多个物理分散在不同broker上的队列，一个Consumer Group包含多个消费者实例，rebalance要做的就是在这些消费者实例间动态划分队列所属权
> 无中心化本地计算模型：所有消费者实例向Broker发送周期性心跳信息，同步获取全局一致的在线实例列表
每个消费者在本地独立运行相同的分配算法，认领属于自己的队列，实现高效的分布式协同

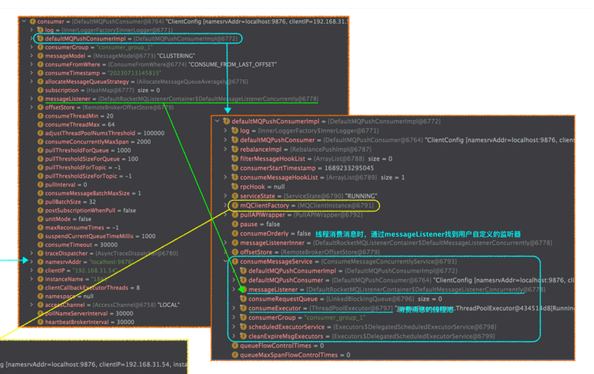
为什么要消费端负责rebalance?
> 去中心化
> 消费端从broker获取「在线的」「消费端列表」
> 各自本地执行同一套分配算法，得出自己负责的队列集合
> 不依赖外部协调，实现消费端之间的「分布式协同」





```
singletonObjects = {ConcurrentHashMap@6342}  size = 84
  "forceAutoProxyCreatorToUseClassProxying" -> {AopAutoConfiguration$ClassProxyingConfiguration$lambda@6659}
  "org.springframework.context.annotation.internalConfigurationAnnotationProcessor" -> {ConfigurationClassPostProcessor@6660}
  "propertySourcesPlaceholderConfigurer" -> {PropertySourcesPlaceholderConfigurer@6661}
  "springApplicationArguments" -> {DefaultApplicationArguments@6663}
  "org.apache.rocketmq.spring.support.DefaultRocketMQListenerContainer_2" -> {DefaultRocketMQListenerContainer@6664} "DefaultRock
  "applicationStartup" -> {DefaultApplicationStartup@6306}
  "methodValidationPostProcessor" -> {FilteredMethodValidationPostProcessor@6666} "proxyTargetClass=true; optimize=false; opaque=f
  "org.apache.rocketmq.spring.support.DefaultRocketMQListenerContainer_1" -> {DefaultRocketMQListenerContainer@6667} "DefaultRock
  "org.springframework.boot.autoconfigure.admin.SpringApplicationAdminJmxAutoConfiguration" -> {SpringApplicationAdminJmxAutoCo
  "org.apache.rocketmq.spring.support.DefaultRocketMQListenerContainer_3" -> {DefaultRocketMQListenerContainer@6669} "DefaultRock
  "jsonComponentModule" -> {JsonComponentModule@6670}
```

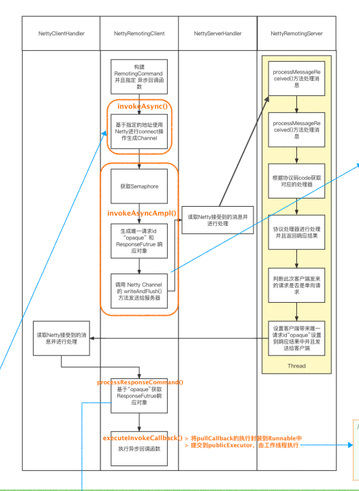
```
value = {DefaultRocketMQListenerContainer@6669} "DefaultRocketMQListenerContainer{consumerGroup='consumer_group_1',
  applicationContext = {AnnotationConfigApplicationContext@5344} "org.springframework.context.annotation.AnnotationConf
  name = "org.apache.rocketmq.spring.support.DefaultRocketMQListenerContainer_3"
  suspendCurrentQueueTimeMillis = 1000
  delayLevelWhenNextConsume = 0
  nameServer = "localhost:9876"
  accessChannel = {AccessChannel@6758} "LOCAL"
  consumerGroup = "consumer_group_1"
  topic = "DEV-TEST"
  consumeThreadMax = 64
  charset = "UTF-8"
  objectMapper = {ObjectMapper@6692}
  messageConverter = {CompositeMessageConverter@6762} "CompositeMessageConverter[converters=[org.springframework.
  rocketMQListener = {Consumers$Consumer1@6739} 用户自定义的消息监听器
  rocketMQMessageListener = {$Proxy40@6763}
  consumer = {DefaultMQPushConsumer@6764} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.54, instance
  messageType = {Class@324} "class java.lang.String"
  running = true
  consumeMode = {ConsumeMode@6765} "CONCURRENTLY"
  selectorType = {SelectorType@6766} "TAG"
  selectorExpression = "*"
  messageModel = {MessageModel@6768} "CLUSTERING"
  consumeTimeout = 30000
```



```
consumer = {DefaultMQPushConsumer@6764} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.
  log = {InnerLoggerFactory$InnerLogger@6771}
  defaultMQPushConsumerImpl = {DefaultMQPushConsumerImpl@6772}
  consumerGroup = "consumer_group_1"
  messageModel = {MessageModel@6773} "CLUSTERING"
  consumeFromWhere = {ConsumeFromWhere@6774} "CONSUME_FROM_LAST_OFFSET"
  consumeTimestamp = "20230713145815"
  allocateMessageQueueStrategy = {AllocateMessageQueueAveragely@6776}
  subscription = {HashMap@6777}  size = 0
  messageListener = {DefaultRocketMQListenerContainer$DefaultMessageListenerConcurrently@6778}
  offsetStore = {RemoteBrokerOffsetStore@6779}
  consumeThreadMin = 20
  consumeThreadMax = 64
  adjustThreadPoolNumsThreshold = 100000
  consumeConcurrentlyMaxSpan = 2000
  pullThresholdForQueue = 1000
  pullThresholdSizeForQueue = 100
  pullThresholdForTopic = -1
  pullThresholdSizeForTopic = -1
  pullInterval = 0
  consumeMessageBatchMaxSize = 1
  pullBatchSize = 32
  postSubscriptionWhenPull = false
  unitMode = false
  maxReconsumeTimes = -1
  suspendCurrentQueueTimeMillis = 1000
  consumeTimeout = 30000
  traceDispatcher = {AsyncTraceDispatcher@6780}
  namesrvAddr = "localhost:9876"
  clientIP = "192.168.31.54"
  instanceName = "184
  clientCallbackExecutorThreads = 8
  namespace = null
  accessChannel = {AccessChannel@6758} "LOCAL"
  pollNameServerInterval = 30000
  heartbeatBrokerInterval = 30000
```

```
defaultMQPushConsumerImpl = {DefaultMQPushConsumerImpl@6772}
  log = {InnerLoggerFactory$InnerLogger@6771}
  defaultMQPushConsumer = {DefaultMQPushConsumer@6764} "ClientConfig [namesrvAddr=localhost:9876, client
  rebalanceImpl = {RebalancePushImpl@6787}
  filterMessageHookList = {ArrayList@6788}  size = 0
  consumerStartTimestamp = 1689233295045
  consumeMessageHookList = {ArrayList@6789}  size = 1
  rpcHook = null
  serviceState = {ServiceState@6790} "RUNNING"
  mQClientFactory = {MQClientInstance@6791}
  pullAPIWrapper = {PullAPIWrapper@6792}
  pause = false
  consumeOrderly = false 拉取消费消息时，通过messageListener找到用户自定义的监听器
  messageListenerInner = {DefaultRocketMQListenerContainer$DefaultMessageListenerConcurrently@6778}
  offsetStore = {RemoteBrokerOffsetStore@6779}
  consumeMessageService = {ConsumeMessageConcurrentlyService@6793}
    defaultMQPushConsumerImpl = {DefaultMQPushConsumerImpl@6772}
    defaultMQPushConsumer = {DefaultMQPushConsumer@6764} "ClientConfig [namesrvAddr=localhost:9876, cl
    messageListener = {DefaultRocketMQListenerContainer$DefaultMessageListenerConcurrently@6778}
    consumeRequestQueue = {LinkedBlockingQueue@6796}  size = 0
    consumeExecutor = {ThreadPoolExecutor@6797} 消费消息的线程池 ThreadPoolExecutor@434514d8[Runnin
    consumerGroup = "consumer_group_1"
    scheduledExecutorService = {Executors$DelegatedScheduledExecutorService@6798}
    cleanExpireMsgExecutors = {Executors$DelegatedScheduledExecutorService@6799}
  queueFlowControlTimes = 0
  queueMaxSpanFlowControlTimes = 0
```

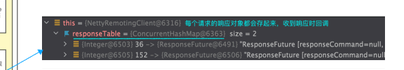
```
mQClientFactory = {MQClientInstance@6791}
  log = {InnerLoggerFactory$InnerLogger@6771}
  clientConfig = {ClientConfig@6805} "ClientConfig [namesrvAddr=localhost:9876, clientIP=192.168.31.54, insta
  instanceIndex = 0
```



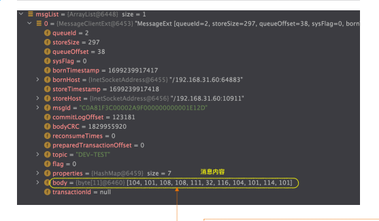
NettyClientHandler
NettyRemotingClient
NettyServerHandler
NettyRemotingServer
构建RemotingCommand 并且指定 异步回调函数
invokeAsync()
基于指定的地址使用Netty进行connect操作生成Channel
获取Semaphore
invokeAsyncAmpl()
生成唯一请求id "opaque" 和 ResponseFuture 响应对象
调用 Netty Channel 的 writeAndFlush() 方法发送给服务器
读取Netty接受到的消息并进行处理
processMessageReceived()方法处理消息
根据协议码code获取对应的处理器
协议处理器进行处理并且返回响应结果
判断此次客户端发来的请求类型是否是单向请求
设置客户端请求来唯一请求id"opaque"设置到响应结果中并且发送给客户端
Thread
读取Netty接受到的消息并进行处理
processResponseCommand()
基于"opaque"获取ResponseFuture响应对象
executeInvokeCallback()
执行异步回调函数

pullCallback是DefaultMQPushConsumerImpl里面内部类实例，持有一个指向外部类实例的字段this$0
可以通过this$0直接访问外部类实例的所有成员，包括private字段和方法

//拉取broker中Topic某个queue的数据，DefaultMQPushConsumerImpl构造pullCallback，MQClientAPIImpl构造RemotingCommand
DefaultMQPushConsumerImpl.pullMessage() -> 检查ProcessQueue容量，判断是否要流控（延迟50ms再拉取消息） -> pullKernelImpl() ->
MQClientAPIImpl.pullMessageAsync(String addr, RemotingCommand request, long timeoutMillis, PullCallback pullCallback) {
remotingClient.invokeAsync(addr, request, timeoutMillis, new InvokeCallback(){…});
}



```
this = {NettyRemotingClient@6316} 每个请求的响应对象都会存起来，收到响应时回调
  responseTable = {ConcurrentHashMap@6363}  size = 2
    {Integer@6503} 36 -> {ResponseFuture@6491} "ResponseFuture [responseCommand=null,
    {Integer@6505} 152 -> {ResponseFuture@6506} "ResponseFuture [responseCommand=null
```



```
msgList = {ArrayList@6448}  size = 1
  0 = {MessageClientExt@6453} "MessageExt [queueId=2, storeSize=297, queueOffset=38, sysFlag=0, born
    queueId = 2
    storeSize = 297
    queueOffset = 38
    sysFlag = 0
    bornTimestamp = 1699239917417
    bornHost = {InetSocketAddress@6455} "/192.168.31.60:64883"
    storeTimestamp = 1699239917418
    storeHost = {InetSocketAddress@6456} "/192.168.31.60:10911"
    msgId = "C0A81F3C00002A9F000000000001E12D"
    commitLogOffset = 123181
    bodyCRC = 1829955920
    reconsumeTimes = 0
    preparedTransactionOffset = 0
    topic = "DEV-TEST"
    flag = 0
    properties = {HashMap@6459}  size = 7          消息内容
    body = {byte[11]@6460} [104, 101, 108, 108, 111, 32, 116, 104, 101, 114, 101]
    transactionId = null
```

Broker返回消息时会根据tag的hash code过滤一次，但有哈希碰撞的可能，所以客户端要再equals比较一次

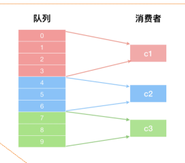
//回调逻辑 > 解码过滤：将borker返回的数据解析成List<MessageExt>，按指定tag二次过滤无用的消息
> case FOUND > msgList缓存到ProcessQueue
> 按指定的消费粒度consumeMessageBatchMaxSize拆分msgList（默认值1，可在application.yml配置），提交给业务逻辑处理（默认是一条一条交）
每条msg封装一个ConsumeRequest，提交到线程池 ConsumeMessageConcurrentlyService.consumeExecutor.submit(consumeRequest)
> pullRequest放回队列，继续拉取消息
> case NO_NEW_MSG or NO_MATCHED_MSG: pullRequest放回队列，继续拉取消息

工作线程执行consumeRequest.run() > 将bytes转成Listener监听的消息类型
> 交给listener消费（用户自定义的Listener监听器）
> 如果业务逻辑抛异常：发回broker重试
> 删除processQueue中缓存的msg
> 更新消费位点offset（定时上报给broker）

```java
void RebalanceImpl.rebalanceByTopic(final String topic, final boolean isOrder) {
    switch (messageModel) {
        case CLUSTERING: {
            Set<MessageQueue> mqSet = topicSubscribeInfoTable.get(topic); //topic下的所有队列
            List<String> cidAll = mQClientFactory.findConsumerIdList(topic, consumerGroup); //消费组的所有消费者实例

            List<MessageQueue> mqAll = new ArrayList<MessageQueue>();
            mqAll.addAll(mqSet);
            Collections.sort(mqAll); //队列排序
            Collections.sort(cidAll); //消费者实例排序
            AllocateMessageQueueStrategy strategy = allocateMessageQueueStrategy;
            //当前实例根据指定策略认领属于自己的队列（当前Consumer Group下的其他实例会根据同样的策略认领其他队列）
            List<MessageQueue> allocateResult = strategy.allocate(consumerGroup, mQClientFactory.getClientId(), mqAll, cidAll);
            Set<MessageQueue> allocateResultSet = new HashSet<MessageQueue>();
            if (allocateResult != null) allocateResultSet.addAll(allocateResult); //去重
            //上报消费位点，给每个新认领的队列生成pullRequest，放到PullMessageService.pullRequestQueue中
            boolean changed = updateProcessQueueTableInRebalance(topic, allocateResultSet, isOrder);
            if (changed) messageQueueChanged(topic, mqSet, allocateResultSet);
            break;
        }
        default: break;
    }
}
```

> Broker维护了Topic信息，并上报给NameServer，客户端和NameServer通信获得Topic完整的路由信息
> Broker维护了Consumer Group信息（实例，订阅关系等），客户端 1).可以发送心跳信息给Broker，将自己添加到Consumer Group中
2).也可以发送请求，获取Consumer Group下的所有实例信息
> 当Topic或者Consumer Group信息变化时，Broker会通知相关客户端rebalance一次

默认使用平均分配策略 > 将topic下的队列排序，将消费者实例按照id排序
> 如果队列数大于实例数：先平均分配，多出来的队列分给排在最前的几台实例
> 否则：排名靠前的实例每台分配一个队列



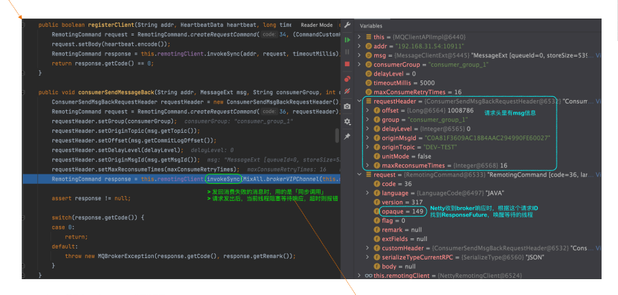
根据负载均衡分配的最新队列结果，消费端实例更新本地缓存的队列processQueueTable ConcurrentMap<MessageQueue, ProcessQueue>
processQueueTable维护了当前消费者实例分配到的所有Broker MessageQueue及其对应的本地缓存队列ProcessQueue
> 移除不再负责的队列：遍历processQueueTable，如果某个MessageQueue不再由当前消费者实例负责
> 为新增队列创建拉取任务：为新增的MessageQueue创建对应的ProcessQueue，计算起始消费位点，构建PullRequest对象，提交给PullMessageService

> Broker上的每个消息队列，都会在消费端有一个对应的本地缓冲队列ProcessQueue
> 本地消息缓存：内部有一个红黑树TreeMap<Long, MessageExt>，从Broker拉取回来、解码、过滤后的消息在消费完之前都缓存在这里
> 消费端限流：防止海量消息击穿消费端内存占用超过阈值（默认1k条/100MB）时，客户端暂停拉取并休眠（默认50ms）

```java
public class PullRequest {
    private String consumerGroup;
    private MessageQueue messageQueue;
    private ProcessQueue processQueue;
    private long nextOffset;
    private boolean lockedFirst = false;
}
```

```java
public void processResponseCommand(ChannelHandlerContext ctx, RemotingCommand cmd) {
    int opaque = cmd.getOpaque();
    ResponseFuture responseFuture = (ResponseFuture)this.responseTable.get(opaque);
    if (responseFuture != null) {
        responseFuture.setResponseCommand(cmd);
        this.responseTable.remove(opaque);
        if (responseFuture.getInvokeCallback() != null) {
            this.executeInvokeCallback(responseFuture);
        } else {…}
    } else {…}
}
```

DefaultRocketMQListenerContainer$DefaultMessageListenerConcurrently.consumeMessage(List<MessageExt> msgs, ConsumeConcurrentlyContext context) {
//遍历msgs，将msg转成自定义监听器的消息类型，然后交给rocketMQListener（也就是用户定义的RocketMQListener实现类）消费消息
try {
DefaultRocketMQListenerContainer.this.rocketMQListener.onMessage(DefaultRocketMQListenerContainer.this.doConvertMessage(messageExt));
} catch (Exception e) { //用户消费消息异常了，
context.setDelayLevelWhenNextConsume(DefaultRocketMQListenerContainer.this.delayLevelWhenNextConsume); //设置重试延迟级别：默认为0（由Broker控制重试频率，根据重试次数递增重试时间）
>0（由客户端控制重试频率）
-1（不重试，直接投入死信队列）
return ConsumeConcurrentlyStatus.RECONSUME_LATER; //重试
}
return ConsumeConcurrentlyStatus.CONSUME_SUCCESS; //成功
}



```java
public boolean registerClient(String addr, HeartbeatData heartbeat, long time
    RemotingCommand request = RemotingCommand.createRequestCommand( code: 34, (CommandCustom
    request.setBody(heartbeat.encode());
    RemotingCommand response = this.remotingClient.invokeSync(addr, request, timeoutMillis)
    return response.getCode() == 0;
}

public void consumerSendMessageBack(String addr, MessageExt msg, String consumerGroup, int
    ConsumerSendMsgBackRequestHeader requestHeader = new ConsumerSendMsgBackRequestHeader()
    RemotingCommand request = RemotingCommand.createRequestCommand( code: 36, requestHeader)
    requestHeader.setGroup(consumerGroup);
    requestHeader.setOriginTopic(msg.getTopic());
    requestHeader.setOffset(msg.getCommitLogOffset());
    requestHeader.setDelayLevel(delayLevel);
    requestHeader.setOriginMsgId(msg.getMsgId());
    requestHeader.setMaxReconsumeTimes(maxConsumeRetryTimes);
    RemotingCommand response = this.remotingClient.invokeSync(MixAll.brokerVIPChannel(this.
                        > 发回消费失败的消息时，用的是「同步调用」
                        > 请求发出后，当前线程阻塞等待响应，超时则抛异常
    assert response != null;

    switch(response.getCode()) {
    case 0:
        return;
    default:
        throw new MQBrokerException(response.getCode(), response.getRemark());
    }
}
```

invokeSync：基于 CountDownLatch 的同步阻塞

虽然 Netty 本身是异步事件驱动的，但 invokeSync 通过以下手段实现了"同步"效果：

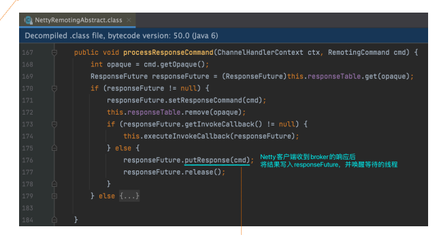
1. 生成唯一 ID：为每个请求生成一个唯一的 opaque（请求 ID）。
2. 创建 Future：创建一个 ResponseFuture 对象，内部包含一个 CountDownLatch(1)，并将该对象以 opaque 为 Key 存入本地缓存 Map。
3. 发送请求：通过 Netty Channel 发送数据。
4. 阻塞等待：调用 responseFuture.waitResponse(timeout)，内部执行 countDownLatch.await()，当前业务线程在此处挂起。
5. 唤醒线程：当 Netty 收到服务端响应时，根据响应中的 opaque 找到对应的 ResponseFuture，调用 countDownLatch.countDown()，唤醒业务线程并返回结果。

注意：如果服务端处理慢或网络抖动，导致等待时间超过设定的 timeoutMillis，会抛出 RemotingTimeoutException。

```java
public RemotingCommand invokeSyncImpl(final Channel channel, final RemotingCommand request, final long timeoutMillis) throws Interru
    final int opaque = request.getOpaque();
    try {
        final ResponseFuture responseFuture = new ResponseFuture(channel, opaque, timeoutMillis, invokeCallback: null, once: null);
        this.responseTable.put(opaque, responseFuture);
        final SocketAddress addr = channel.remoteAddress();
        channel.writeAndFlush(request).addListener((…)); 将request写入Netty的发送缓冲区，并触发底层Socket写出

        RemotingCommand responseCommand = responseFuture.waitResponse(timeoutMillis); 当前线程挂起等待broker的响应结果

        if (null == responseCommand) {…}
        return responseCommand;
    } finally {…}
}
```

```java
public RemotingCommand waitResponse(long timeoutMillis) throws InterruptedException {
    this.countDownLatch.await(timeoutMillis, TimeUnit.MILLISECONDS); 1.计数器AQS.state不为0时，当前线程将自己加入同步队列，挂起等待
                                                                    2.线程被唤醒后，检查AQS.state，发现为0，结束等待，继续往下执行
    return this.responseCommand;
}
```

```java
public class ResponseFuture {
    private final int opaque;
    private final Channel processChannel;
    private final long timeoutMillis;
    private final InvokeCallback invokeCallback;
    private final long beginTimestamp = System.currentTimeMillis();
    private final CountDownLatch countDownLatch = new CountDownLatch(1); 创建ResponseFuture时，CountDownLatch计数器设为1
```



NettyRemotingAbstract.class

Decompiled .class file, bytecode version: 50.0 (Java 6)

```java
public void processResponseCommand(ChannelHandlerContext ctx, RemotingCommand cmd) {
    int opaque = cmd.getOpaque();
    ResponseFuture responseFuture = (ResponseFuture)this.responseTable.get(opaque);
    if (responseFuture != null) {
        responseFuture.setResponseCommand(cmd);
        this.responseTable.remove(opaque);
        if (responseFuture.getInvokeCallback() != null) {
            this.executeInvokeCallback(responseFuture);
        } else {
            responseFuture.putResponse(cmd);   Netty客户端收到broker的响应后
                                               将结果写入responseFuture，并唤醒等待的线程
            responseFuture.release();
        }
    } else {…}
}
```

```java
public void putResponse(RemotingCommand responseCommand) {
    this.responseCommand = responseCommand;
    this.countDownLatch.countDown(); > AQS.state 值减1
                                     > 检查 state 值，如果为0，唤醒同步队列中排在首位的线程
                                     > 同步队列的线程会像多米诺骨牌一样，依次被排在前面的线程唤醒
}
```